You’ve got a really clear picture of what you want—this is less “dumb little editor” and more “focused, si

High■level comparison

Option	JSON handling	DDS display	Packaging as single folder e
Node.js + NW.js	Excellent (native)	Needs DDS decoder	Yes (nw.exe + app folder)
C++ + Irrlicht	Manual / library JSON	Native DDS support	Yes (exe + dlls)
C++ + SDL2 + OpenGL	Manual / library JSON	Needs DDS decoder	Yes

---

Direct recommendation

Given everything you wrote—your comfort with C++, your desire to keep it small, offline, and self■containe

> **C++ + Irrlicht + a JSON library (e.g., nlohmann/json)**

Reasons:

- **DDS is “just works”** in Irrlicht; no need to fight Chromium/WebGL or custom decoders.
- **You already know Irrlicht and OpenGL concepts**, so UI + 2D sprites + text is straightforward.
- **No browser stack, no NW.js runtime**, just an `.exe` and a few `.dll`s in your mod folder.
- JSON parsing in C++ is easy with a good library; the hard part is your data model, not the language.

If you really want to stay in JavaScript, NW.js is still viable—but you’ll pay a complexity tax for DDS.

Option 1: C++ + Irrlicht editor

Core pieces

- **Rendering/UI:**
  - Use Irrlicht in a simple 2D GUI style:
    - One main window.
    - Left panel: list of vanilla mechs (parsed from game JSON).
    - Middle/right panel: current custom mech (slots, weapons, equipment).
    - Side/preview area: DDS portrait/thumbnail of the mech.
  - Use Irrlicht’s GUI elements (`IGUIEnvironment`, list boxes, edit boxes, buttons) for a basic editor UI

- **JSON handling:**
  - Pull in **nlohmann/json** (single header) or similar.
  - Build small C++ structs for:
    - `WeaponDef`, `AmmoBoxDef`, `PilotDef`, `MechDef`, `ChassisDef`, `MoveDef`, etc.
  - Write:
    - **Load functions**: parse vanilla JSON + your mod JSON into those structs.
    - **Save functions**: serialize your structs back into ModTek■compatible JSON.

- **Data flow:**
  1. On startup, scan:
     - Vanilla game data folders (for base mech/chassis/move defs).
     - Your mod folder (for custom weapons/equipment/etc.).
  2. Build in■memory catalogs:
     - “Available weapons”, “available equipment”, “available chassis”, etc.
  3. UI lets you:
     - Pick a base vanilla mech.
     - Clone it into a “custom mech” definition.
     - Assign weapons/equipment from your custom catalog.
  4. Save to:
     - `mod_folder/CustomMechs/mechdef_*.json`
     - `mod_folder/CustomMechs/chassisdef_*.json`
     - `mod_folder/CustomMechs/movedef_*.json` (if needed).

- **Packaging:**
  - Put in your mod folder:
    - `Editor/`
      - `bt_mech_editor.exe`
      - `Irrlicht.dll` (from MSYS2)
      - `json.hpp` (in `src/` only)
      - `config.json` (paths to game + mod)